Online Multi-Restaurant Delivery System

PROJECT DOCUMENTATION

Prepared By :

Name	ID
Mariam Hafez Saed	23011150
Kholoud Hany Mahmoud	23011071
Joly Ahmed Abd El-Hafez	23011245
Mariam Khaled Ahmed	23011528
Jasmine Mohammed Abd El-Menaiem	23011058

Online Multi-Restqurant Delivery System

Problem Description

Traditional food ordering methods often rely on direct phone calls or physical visits, which present several challenges in today's fast-paced environment. The primary issues addressed by this system include:

- **Operational Inefficiency:** Manual order taking is prone to human error, leading to incorrect orders, miscommunication of special requests, and lost revenue for restaurants.
- **Lack of Transparency:** Customers often face uncertainty regarding the status of their orders. Without real-time tracking, there is no visibility into whether food is being prepared or is currently in transit.
- **Logistical Fragmentation:** Coordinating between available delivery drivers and multiple restaurants is difficult without an automated central system, leading to longer delivery times and cold food.
- **Limited Market Reach:** Many small-to-medium restaurants lack the digital infrastructure to reach a wider customer base beyond their immediate physical vicinity.
- **Payment & Data Security:** Handling cash transactions or sharing credit card details over the phone poses security risks for both customers and business owners.

System Objectives

The goal of this project is to develop a robust digital ecosystem that:

1. **Streamlines the Ordering Process:** By providing a user-friendly interface to browse, customize, and place orders in few taps.
2. **Enhances Visibility:** Through a real-time tracking system that updates the order status from "Pending" to "Delivered."
3. **Optimizes Logistics:** By implementing an automated driver assignment algorithm based on proximity and rating.
4. **Ensures Reliability:** By maintaining 99.9% availability and high performance during peak hours (e.g., weekends and holidays).
5. **Secures Transactions:** By integrating third-party payment gateways and ensuring data encryption for all user information.

Stakeholders

To ensure the system meets all business and technical needs, we have identified the following key stakeholders:

- **Customers (End-Users):** The primary users who interact with the app to browse menus, place orders, make payments, and track their deliveries. Their main interest is ease of use, speed, and order accuracy.
- **Restaurant Owners & Staff:** Users who manage their digital storefront, update menu availability, and process incoming orders via the restaurant dashboard.
- **Delivery Personnel (Drivers):** Independent contractors or employees who use the driver-side app to accept delivery tasks, navigate to locations, and update order statuses in real-time.
- **System Administrators (Managers):** The internal team responsible for monitoring the entire platform, handling disputes, managing restaurant partnerships, and overseeing system maintenance.
- **Payment Gateway Providers:** External third-party entities (e.g., Stripe, PayPal, or local banks) that facilitate secure financial transactions between the customer and the platform.

Requirements Engineering:

1. Functional Requirements :

FR-01 – User Registration & Login:

User Requirement : A customer shall be able to create a new account by providing their full name, email address, and password, and shall be able to log in using their credentials to access the platform.

System Requirement : The system shall validate the email format and check it is not already registered. It shall hash the password before storing it. Upon successful login, the system shall generate a session token to maintain the authenticated state.

Priority : High

Actor : Customer

Rationale : This is the entry point for all system functionality. Without authentication, no personalized features can be accessed.

Pre-condition : The user has the app installed and is not currently logged in.

Post-condition : The user is authenticated and redirected to the home screen.

Normal Flow :

- User opens the app and selects "Sign Up" or "Log In"
- User enters their details and submits the form
- System validates the input
- System creates the account or verifies credentials
- System grants access and loads the home screen

Alternative Flow : If the email is already registered, the system displays an error and prompts the user to log in instead. If credentials are incorrect, the system allows the user to retry or reset their

password.

FR-02 – Browse Restaurants

User Requirement : A customer shall be able to browse a list of available restaurants near their location and filter results by cuisine type, rating, or estimated delivery time.

System Requirement : The system shall retrieve the user's GPS coordinates and query the database for restaurants within a configurable delivery radius. It shall apply the selected filters and return a sorted list of matching restaurants.

Priority : High

Actor : Customer

Rationale : Browsing is the first step in the ordering process. Without it, customers cannot discover available options.

Pre-condition : The user is logged in and has location access enabled.

Post-condition: A filtered list of nearby restaurants is displayed to the user.

Normal Flow :

- User opens the home screen
- System detects user location automatically
- System fetches and displays nearby restaurants
- User applies filters if desired
- System refreshes the list based on selected filters

Alternative Flow : If location access is denied, the system prompts the user to enter their address manually. If no restaurants match the filters, the system displays a "No results found" message and suggests removing filters.

FR-03 – View Restaurant Menu

User Requirement : A customer shall be able to select a restaurant and view its full menu, including item names, descriptions, prices, and availability.

System Requirement : The system shall fetch the menu data associated with the selected restaurant from the database and render it grouped by category. Items marked as unavailable by the restaurant shall be displayed as greyed out and non-selectable.

Priority : High

Actor : Customer

Rationale : Viewing the menu is essential before any order can be placed. Customers need complete and accurate item information to make purchasing decisions.

Pre-condition : The user has selected a restaurant from the browse screen.

Post-condition: The restaurant's menu is fully displayed with current availability status.

Normal Flow :

- User taps on a restaurant from the list
- System retrieves the menu from the database
- System displays items grouped by category (e.g. Appetizers, Main Course, Drinks)
- User browses items and views details

Alternative Flow : If the menu fails to load due to a network error, the system displays an error message and offers a retry option. If the restaurant has temporarily closed, the system notifies the user and disables the ordering option.

FR-04 – Manage Shopping Cart

User Requirement : A customer shall be able to add items to a cart, adjust quantities, remove items, and view the total price before placing an order.

System Requirement : The system shall maintain a session-based cart that stores item IDs, selected quantities, unit prices, and a calculated running total including any applicable fees or discounts. The cart shall persist across screens during the same session.

Priority : High

Actor : Customer

Rationale : The cart acts as the bridge between browsing and ordering. It allows customers to review and confirm their selection before committing to a purchase.

Pre-condition : The user has viewed a restaurant menu and selected at least one item.

Post-condition: The cart reflects the correct items, quantities, and total price.

Normal Flow :

- User taps "Add to Cart" on a menu item
- System adds the item and updates the running total
- User adjusts quantities or removes items as needed
- System recalculates the total after each change
- User proceeds to checkout when satisfied

Alternative Flow : If the user attempts to add items from a different restaurant while the cart already contains items, the system warns the user and asks whether to clear the existing cart or cancel the action.

FR-05 – Place Order

User Requirement : A customer shall be able to place an order from their cart and receive an order confirmation along with an estimated delivery time.

System Requirement : The system shall validate that all cart items are still available, create an order record in the database with a unique order ID, calculate the estimated delivery time, and send a confirmation notification to the customer. It shall also notify the restaurant of the new order.

Priority : High

Actor : Customer, System, Restaurant

Rationale : Placing an order is the core transaction of the entire system. All other features support or follow from this action.

Pre-condition : The user has a non-empty cart and has completed the payment step.

Post-condition : An order record is created, the restaurant is notified, and the customer receives a confirmation.

Normal Flow :

- User reviews cart and taps "Place Order"
- System validates item availability
- System processes the payment
- System creates the order record and assigns a unique ID
- System sends confirmation to the customer with estimated delivery time
- System notifies the restaurant

Alternative Flow : If an item becomes unavailable between cart addition and order placement, the system alerts the user and removes the item from the cart before proceeding. If payment fails, the order is not created and the user is prompted to try again.

FR-06 – Real-Time Order Tracking

User Requirement : A customer shall be able to track the status of their order in real time after it has been placed, from preparation through to delivery.

System Requirement : The system shall maintain and update an order status field that progresses through defined states: Confirmed, Preparing, Ready for Pickup, Out for Delivery, and Delivered. Status changes shall be pushed to the customer's interface without requiring a manual refresh.

Priority : High

Actor : Customer, System, Driver

Rationale : Real-time tracking increases customer trust and reduces the need for support inquiries about order status.

Pre-condition : The customer has successfully placed an order.

Post-condition : The customer can see the current status of their order updated in real time.

Normal Flow :

- Customer opens the order tracking screen
- System displays the current order status
- Restaurant updates status to "Preparing"
- Driver updates status to "Out for Delivery"
- System marks the order as "Delivered" upon completion

Alternative Flow : If the driver is delayed, the system updates the estimated delivery time accordingly and notifies the customer. If the connection is lost, the system displays the last known status and attempts to reconnect automatically.

FR-07 – Payment Processing

User Requirement : A customer shall be able to pay for their order using a credit card, debit card, or digital wallet, and shall receive a payment confirmation upon success.

System Requirement : The system shall integrate with a third-party payment gateway to securely process transactions. It shall not store raw card details and shall use tokenization for payment data. A payment receipt shall be generated and sent to the user's registered email upon successful transaction.

Priority : High

Actor : Customer, System

Rationale : Payment processing is a critical step in completing any order. The system must handle it securely and reliably to protect user financial data.

Pre-condition : The user has reviewed their cart and is ready to confirm the order.

Post-condition : The payment is processed, a receipt is issued, and the order proceeds to the restaurant.

Normal Flow :

- User selects a payment method at checkout
- User enters payment details or selects a saved method
- System sends the transaction to the payment gateway
- Payment gateway returns a success response
- System confirms the order and sends a receipt to the user

Alternative Flow : If the payment gateway returns a failure response, the system notifies the user with the reason and allows them to retry with the same or a different payment method. If a timeout occurs, the system informs the user and does not create an order record.

FR-08 – Restaurant Dashboard & Menu Management

User Requirement : A restaurant owner shall be able to log in to a dedicated dashboard where they can add, edit, or remove menu items and mark items as available or unavailable.

System Requirement : The system shall provide a restaurant-facing interface with role-based

access control. Changes made to the menu shall be written to the database immediately and reflected in the customer-facing app in real time.

Priority : Medium

Actor : Restaurant Owner

Rationale : Restaurants need to maintain accurate and up-to-date menus. Without this feature, customers may order unavailable items, leading to cancellations and poor experience.

Pre-condition : The restaurant owner has a verified account with restaurant-level permissions.

Post-condition : Menu changes are saved and immediately visible to customers browsing the restaurant.

Normal Flow :

- Restaurant owner logs in to the dashboard
- Owner navigates to the menu management section
- Owner adds a new item, edits an existing one, or marks one as unavailable
- System saves the changes to the database
- Updated menu is immediately visible in the customer app

Alternative Flow : If required fields such as item name or price are missing, the system displays a validation error and prevents saving. If the session expires, the owner is redirected to the login page without losing unsaved changes.

FR-09 – Delivery Driver Assignment

User Requirement : Once an order is ready for pickup, the system shall automatically assign an available delivery driver to collect and deliver the order.

System Requirement : The system shall query the location data of all currently active and unassigned drivers. It shall calculate proximity to the restaurant and assign the nearest available driver. The driver shall receive a push notification with the order and pickup details.

Priority : High

Actor : System, Driver

Rationale : Automated driver assignment reduces manual coordination overhead and ensures timely delivery by selecting the most geographically suitable driver.

Pre-condition : The restaurant has confirmed the order is ready for pickup and at least one driver is available.

Post-condition: A driver is assigned to the order and notified with full pickup details.

Normal Flow :

- Restaurant marks the order as "Ready for Pickup"

- System queries all available drivers and their current locations
- System selects the nearest driver
- System sends a push notification to the assigned driver
- Driver accepts the assignment and proceeds to the restaurant

Alternative Flow : If no drivers are available, the system retries every 60 seconds and notifies the customer of the delay. If the assigned driver rejects the order, the system reassigns to the next nearest available driver.

FR-10 – Ratings & Reviews

User Requirement : A customer shall be able to rate a restaurant on a scale of 1 to 5 stars and optionally write a text review after an order has been delivered.

System Requirement : The system shall allow exactly one review submission per completed order. Upon submission, it shall store the rating and review text, update the restaurant's overall average rating, and display the review on the restaurant's public profile.

Priority : Medium

Actor : Customer

Rationale : Ratings and reviews help customers make informed choices and provide restaurants with feedback to improve their service quality.

Pre-condition : The order status has been marked as "Delivered."

Post-condition : The review is saved, the restaurant's average rating is updated, and the review appears on the restaurant's profile.

Normal Flow :

- Customer receives a prompt to rate the order after delivery
- Customer selects a star rating and optionally writes a review
- Customer submits the review
- System stores the review and recalculates the restaurant's average rating
- Review is published on the restaurant's profile page

Alternative Flow : If the customer dismisses the prompt, they can still access the review option from their order history. If the customer attempts to submit a review for the same order twice, the system rejects the second submission and displays a message indicating a review already exists.

FR-11 – Order History & Reorder

User Requirement : A customer shall be able to view a complete history of their past orders and quickly reorder items from a previous purchase with a single action.

System Requirement : The system shall retrieve all completed order records linked to the authenticated user's account, sorted by date in descending order. When the reorder action is triggered, the system shall populate the cart with the same items from the selected past order, subject to current availability.

Priority : Medium

Actor : Customer

Rationale : Order history improves convenience and encourages repeat usage by reducing the effort required to place recurring orders.

Pre-condition : The user is logged in and has at least one completed order.

Post-condition : The selected past order's items are added to the cart, ready for checkout.

Normal Flow :

- User navigates to the "Order History" section
- System retrieves and displays all past orders sorted by date
- User selects a past order to view its details
- User taps "Reorder"
- System adds available items to the cart and notifies the user if any items are no longer available

Alternative Flow : If all items from the past order are unavailable, the system informs the user and does not populate the cart. If the restaurant from the past order is no longer active on the platform, the system displays a message and suggests similar alternatives.

2. Non-Functional Requirements :

1. Security

data must be transmitted via HTTPS and use TLS encryption

- Securing the payment process by having PCI-DSS compliance when handling cards
- Protect user data either by using password hashing, token-based authentication or encryption to prevent data leakage
- Prevent common attacks like SQL injection, XSS

2.Performance & Scalability

- App preferably to load the main screens within from 2–3 seconds
- Handle consurency in ordering or payment or usage in general during peak times (like offers or eid or weekends)
- Order placement latency should be near real-time no delaying as possible
- System must scale horizontally using cloud auto-scaling

3.Availability & Reliability

- System uptime of more than 99.9%
- Graceful handling of failures by doing retry mechanisms for orders/payments
- No order duplication or loss
- Backup and disaster recovery mechanism
- when server down should return up again in range of 2 hours to maximum of 1 day

3. Usability

- Intuitive UI for customers, drivers, and restaurants easy to access for them and in their languages they understand
- Minimal steps to place an order ideally ≤ 5 taps
- Accessible design simple and easy to understand
- support for different languages
- readable fonts
- Consistent experience across device
- Push notifications delivered within seconds
- No duplicate notifications
- Reliable delivery even under load

4.Compatibility & Portability

- Works on:
  - Android & iOS devices
  - Different screen sizes
  - desktop
- Backward compatibility with older OS versions
- Web version support

5.Network & Offline Handling

- Graceful degradation on slow networks and decleration of slowing or dropping in network
- Show cached data if connection drops
- Retry retrieving failed or incomplete requests automatically

6. Maintainability

- microservices Modular code architecture
- Easy to update menus, pricing, and promotions
- Logging and monitoring for debugging
- Clear API documentation

7. Data Consistency & Integrity

- Payment and order status must match the req to implement
- Real-time synchronization between:
  - Customer app
  - Restaurant system
  - Driver app
  - manager report

8. Real-Time Tracking

- Location updates every few seconds
- Accurate ETA calculation

- Efficient use of GPS and battery

9. Scalability of Services

- Handle spikes during promotions or holidays
- Queue management for high order volumes
- Load balancing across servers

11. Localization

- Multi-language support (e.g., Arabic & English)
- Local currency and payment methods
- Region-specific restaurant listings

12. Compliance Legality

- GDPR or local data protection laws
- Secure storage of user and transaction data
- Terms of service & privacy policy enforcement

Requirements Analysis (Ambiguity & Conflicts)

In this phase, we analyzed the collected requirements to identify and resolve any potential ambiguities or contradictions. This ensures the system is logical, consistent, and technically feasible.

Ambiguity Resolution

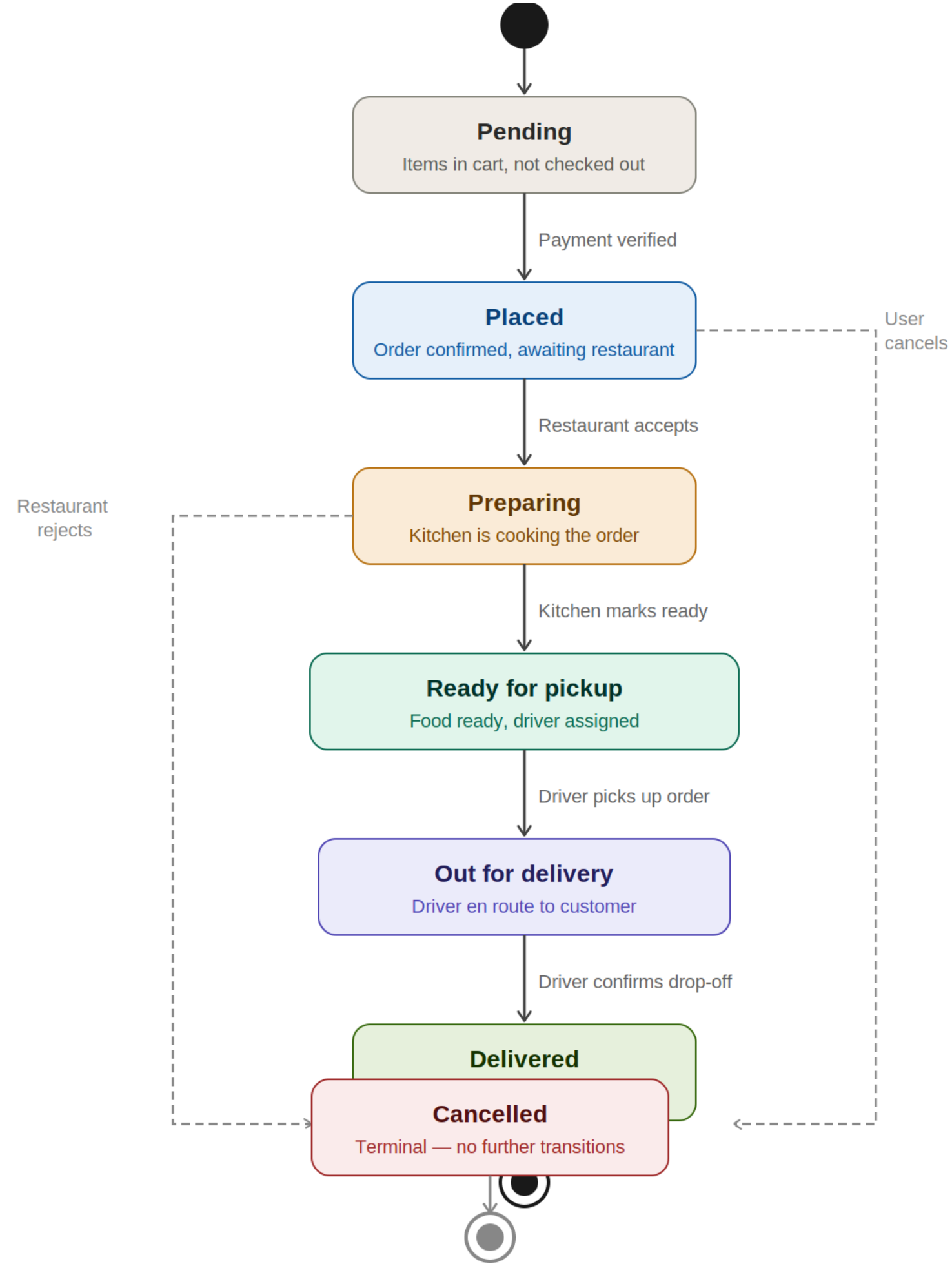
- Ambiguity 1: Real-Time Tracking Precision (Ref: FR-06)
  - Issue: The term "Real-time" is technically vague. Does it mean millisecond updates or minute-by-minute?
  - Resolution: We defined "Real-time" as GPS location updates sent every 5 seconds, with status changes reflected on the user interface within a maximum latency of 2 seconds.
- Ambiguity 2: Proximity for "Nearby" Restaurants (Ref: FR-02)
  - Issue: "Nearby" varies based on urban density and delivery logistics.
  - Resolution: We specified a default search radius of 10km, which can be dynamically adjusted by the system administrator based on driver availability in specific zones.

Conflict Resolution

- Conflict 1: High Security vs. Seamless Usability (Ref: FR-01 & Usability)
  - Issue: Strong security (multi-factor authentication/re-entering card details) conflicts with the goal of placing an order in under 5 taps.
  - Resolution: We implemented Secure Tokenization for payments. This allows users to pay with saved "tokens" (One-tap payment) while the actual sensitive data is handled securely by a third-party PCI-DSS compliant gateway.
- Conflict 2: Driver Proximity vs. Service Quality (Ref: FR-09)

- Issue: Assigning the "Nearest Driver" might lead to a poor experience if that driver has a low rating.
- Resolution: The assignment algorithm was refined to prioritize drivers based on a weighted score of both Proximity and Rating, ensuring the fastest delivery without compromising service quality.

State Machine Diagram (Bonus)



Model Explanation

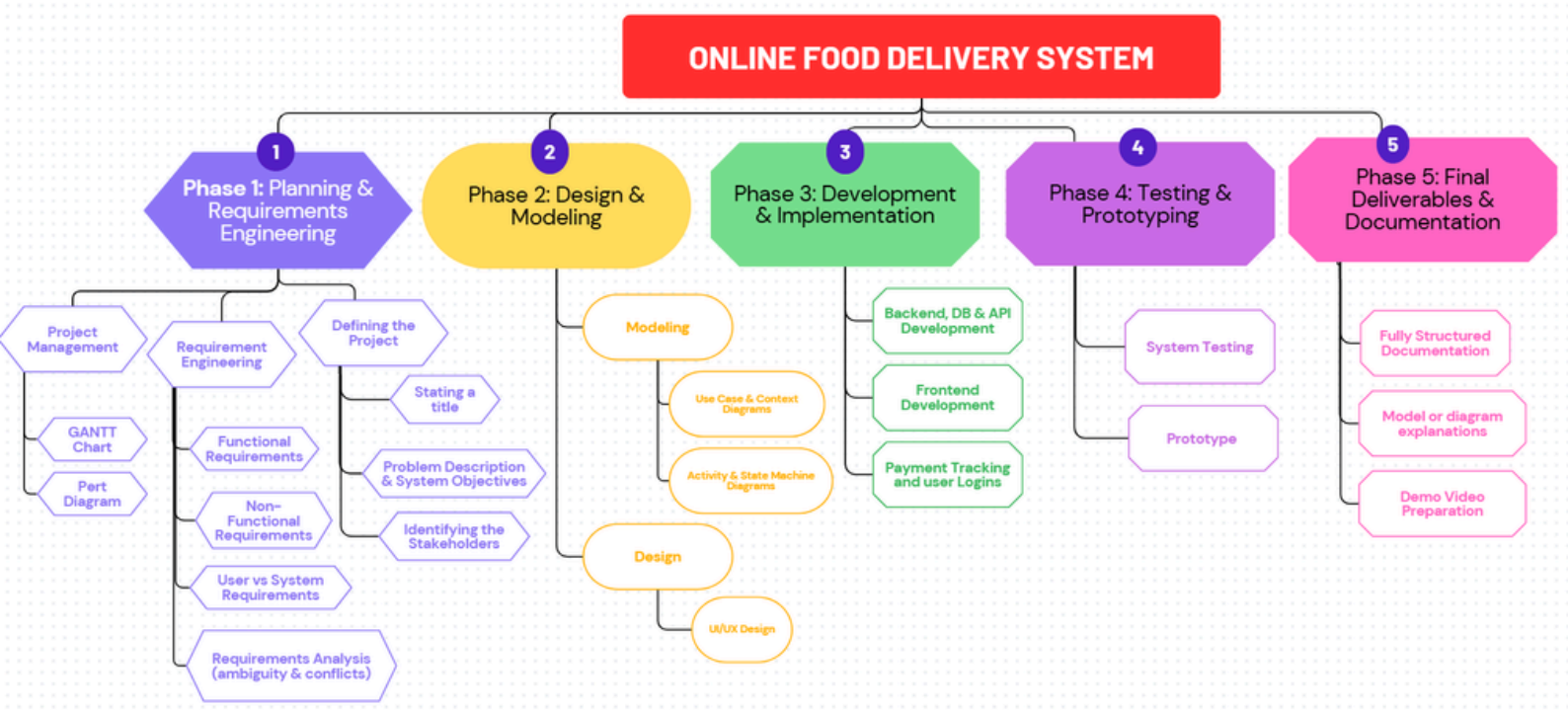
- Purpose: The State Machine Diagram is used to model the lifecycle of the Order Object. It focuses on the various "States" an order transitions through, ensuring the system handles every possible status change correctly without data loss.
- How it Connects:
  - Connection to Activity Diagram: The "States" in this model are the direct results of the "Actions" performed in the System and Driver swimlanes of the Activity Diagram.
  - Connection to Non-Functional Requirements: It ensures Data Consistency, as it provides a single source of truth for the order status across the Customer, Restaurant, and Driver applications.

Order Lifecycle States:

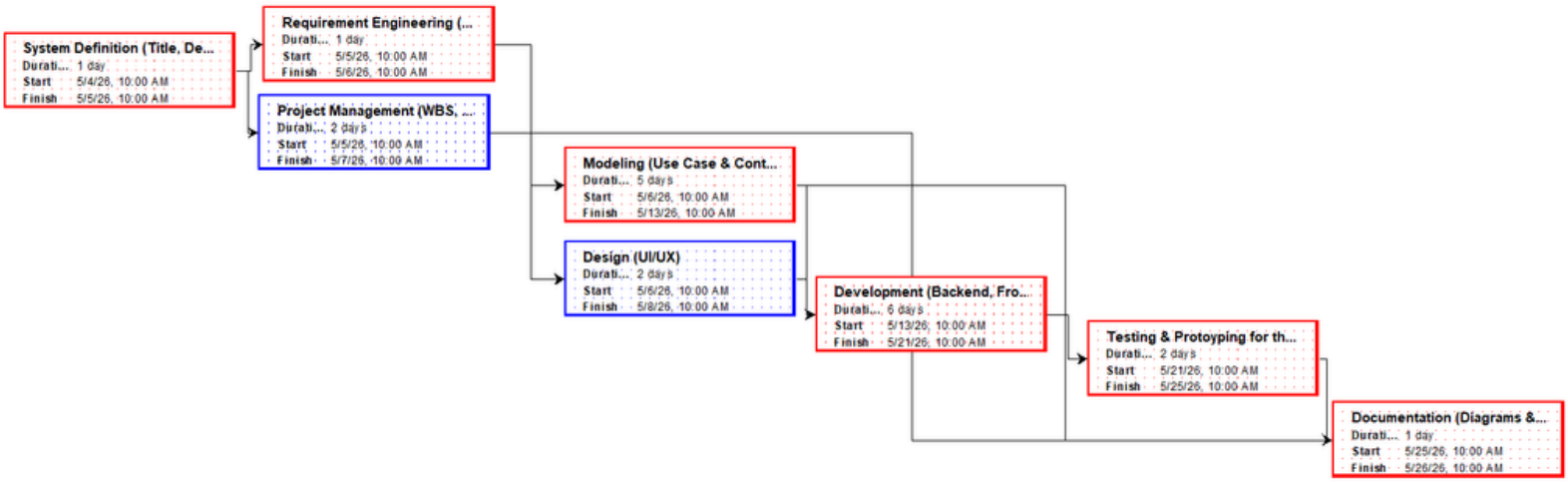
- Pending: The initial state when a user adds items to the cart but hasn't checked out.
- Placed: Transition occurs once the payment is successfully verified.
- Preparing: State enters when the restaurant accepts the order via their dashboard.
- Ready for Pickup: The kitchen marks the food as ready.
- Out for Delivery: Transition occurs when the assigned driver picks up the order.
- Delivered: The final state once the driver confirms the drop-off.
- Cancelled: A terminal state that can be reached if the user cancels (before preparation) or the restaurant rejects the order.

Project Planning:

1. WBS:

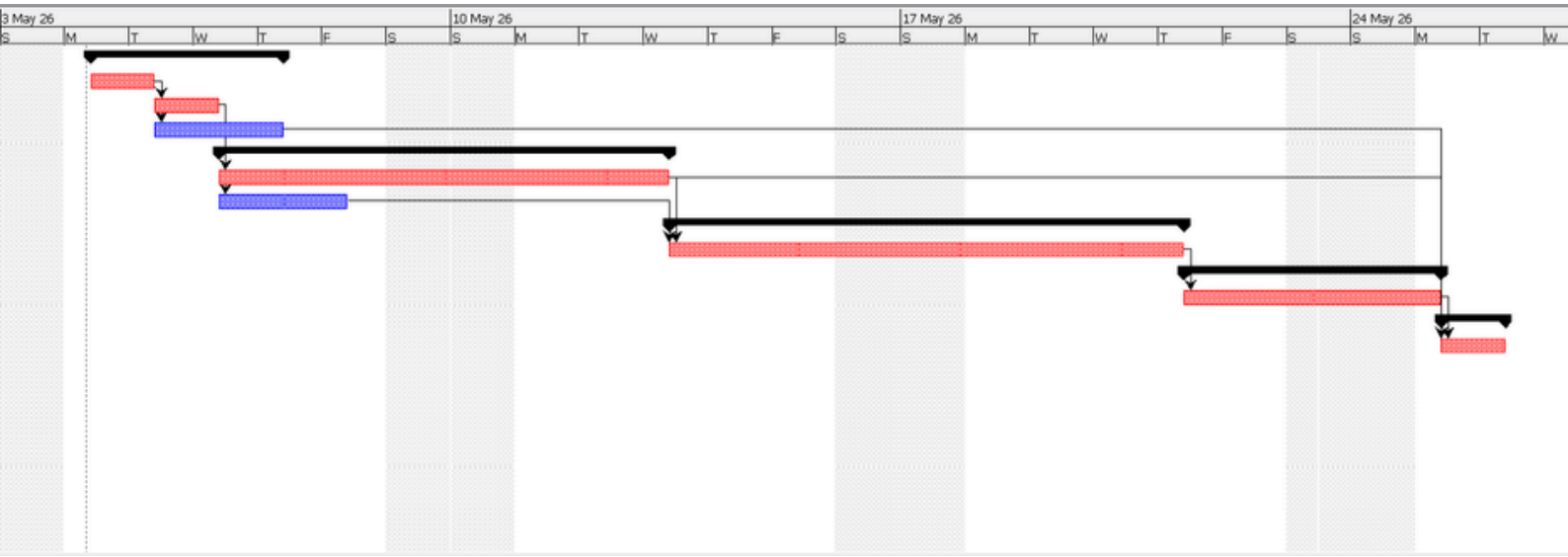


2. Pert Diagram



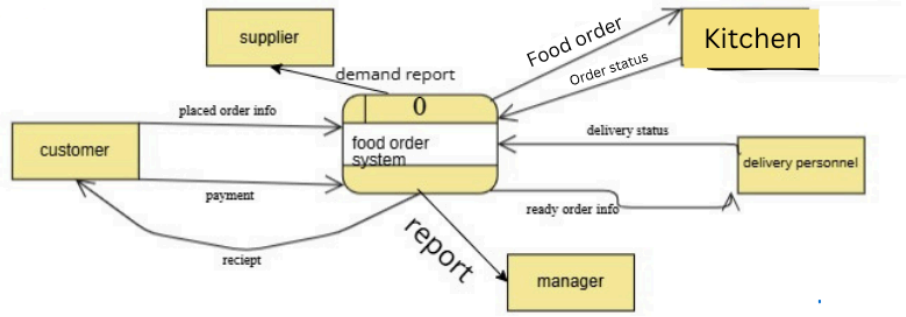
3. Gantt Chart

Name	Duration	Start	Finish	Predecessors
Phase 1: Planning & Requirement Engineering	3 days	**5/4/26, 10:00 A...**	**5/7/26, 10:00 ...**	
System Definition (Title, Description & Objectives, Stakeholders)	1 day	5/4/26, 10:00 AM	5/5/26, 10:00 AM	
Requirement Engineering (Functional, Non Functional, User Vs System, Analysis	1 day	5/5/26, 10:00 AM	5/6/26, 10:00 AM	2
Project Management (WBS, GANTT, Pert)	2 days	5/5/26, 10:00 AM	5/7/26, 10:00 AM	2
Phase 2: Design & Modeling	5 days	**5/6/26, 10:00 A...**	**5/13/26, 10:00 ...**	
Modeling (Use Case & Context, Activity & State Machine Modeling)	5 days	5/6/26, 10:00 AM	5/13/26, 10:00 AM	3
Design (UI/UX)	2 days	5/6/26, 10:00 AM	5/8/26, 10:00 AM	3
Phase 3: Development & Implementation	6 days	**5/13/26, 10:00 ...**	**5/21/26, 10:00 ...**	
Development (Backend, Frontend & DB, API Integration)	6 days	5/13/26, 10:00 AM	5/21/26, 10:00 AM	6;7
Phase 4: Testing & Prototyping	2 days	**5/21/26, 10:00 ...**	**5/25/26, 10:00 ...**	
Testing & Protoyping for the system	2 days	5/21/26, 10:00 AM	5/25/26, 10:00 AM	9
Phase 5: Final Deliverables & Documentation	1 day	**5/25/26, 10:00 ...**	**5/26/26, 10:00 ...**	
Documentation (Diagrams & Demo Video)	1 day	5/25/26, 10:00 AM	5/26/26, 10:00 AM	4;6;11



System Modeling:

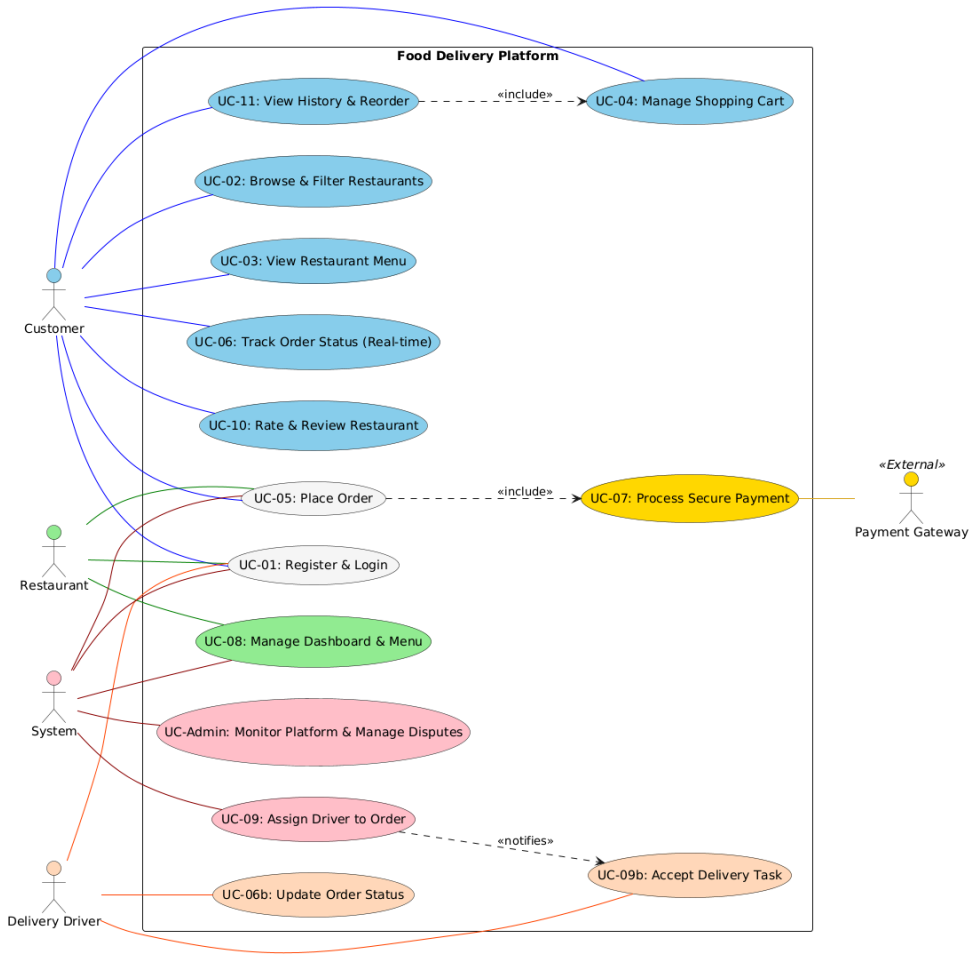
1. Context Diagram



Model Explanation:

- defines what the software is responsible for and what it is not
- identifies exactly who or what will interact with the system.
- The Customer: Who browses menus and places orders.
- The Restaurant: Which receives order details and updates status.
- Delivery Personnel: Who receive pickup and drop-off instructions.
- Payment Processor: A third party (like Stripe or PayPal) that handles the transaction.
- supplier: who supplies the system with new restaurants or with campaigns
- manager : who manages the application

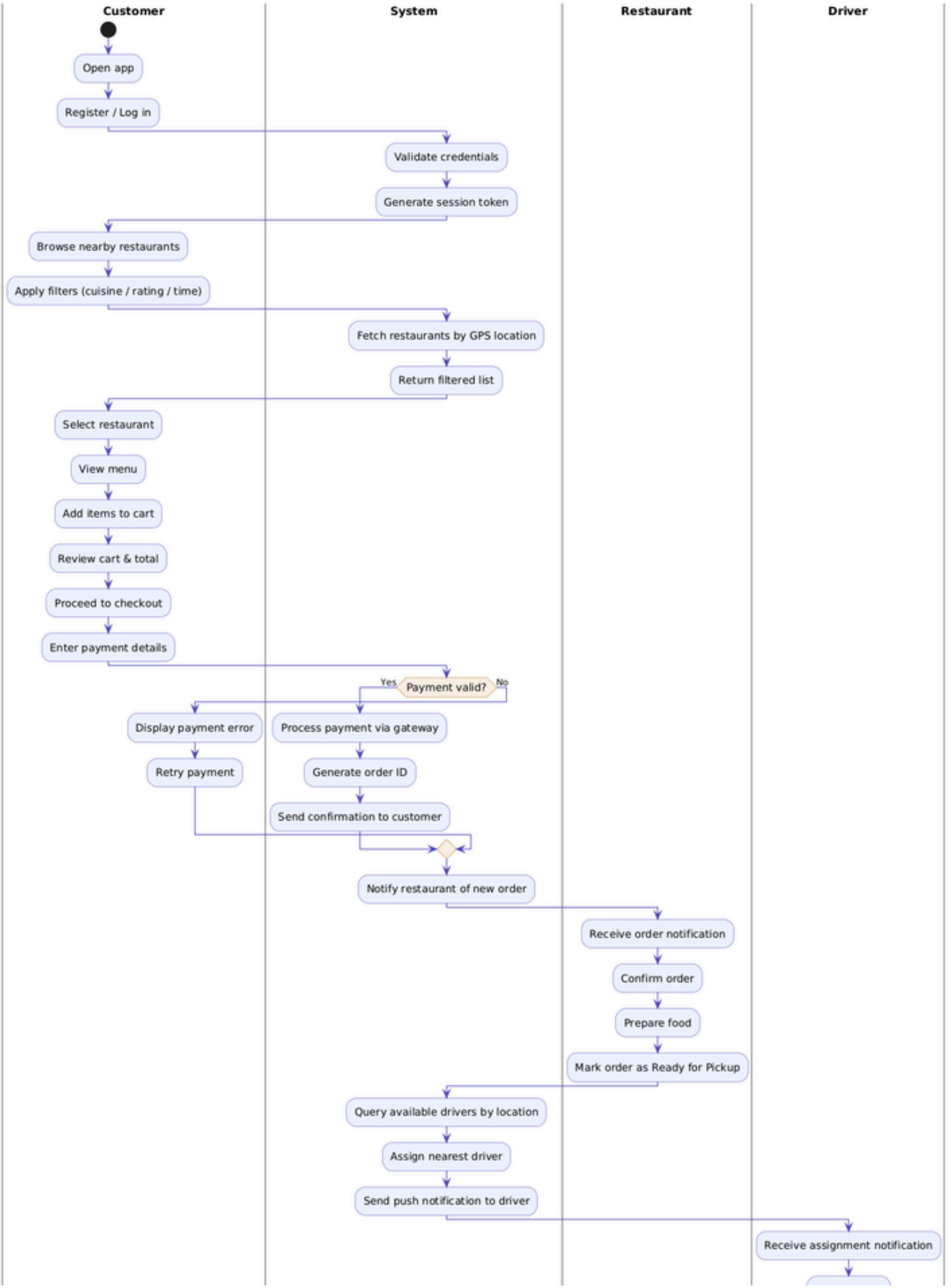
2. Use Case Diagram

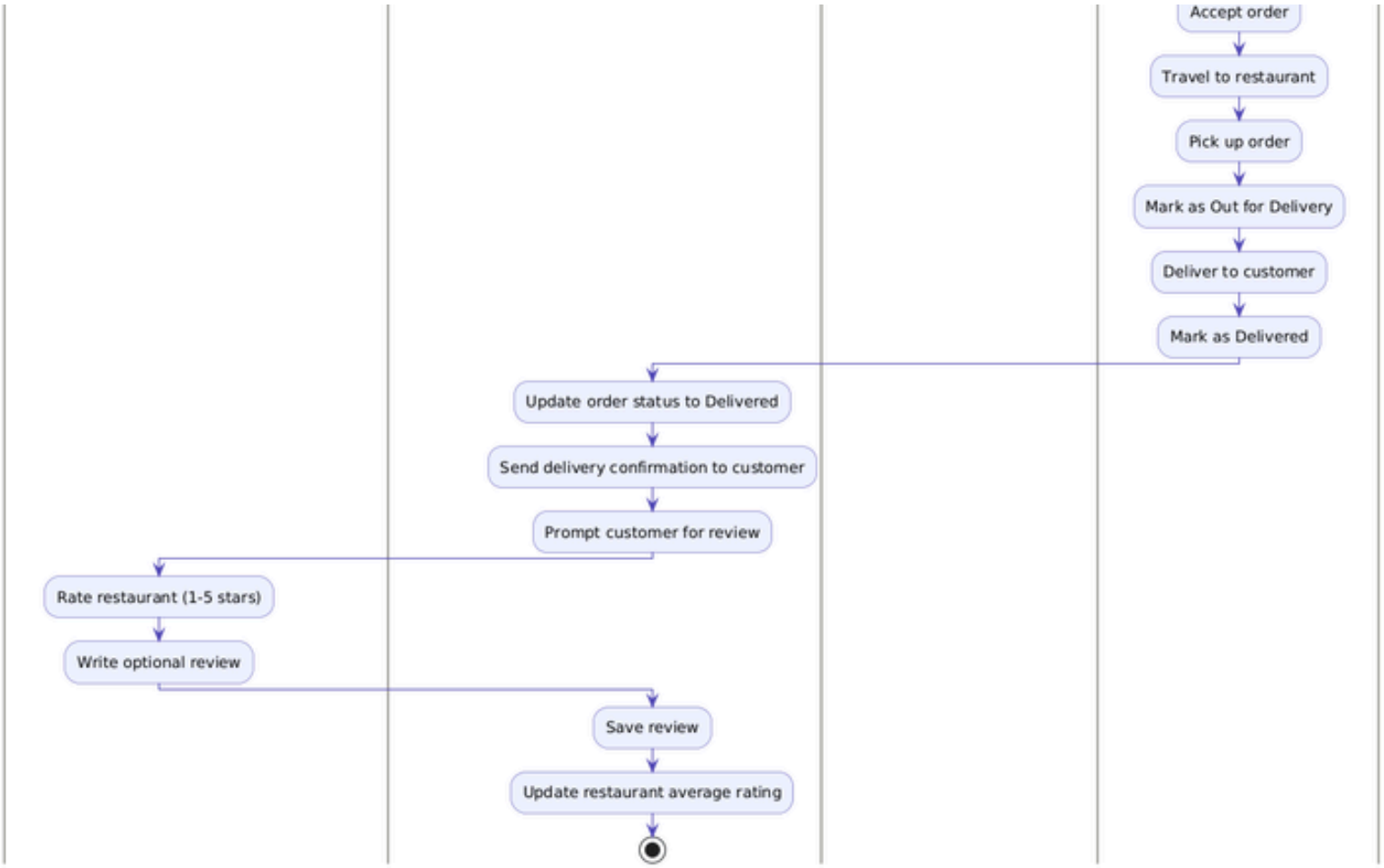


Model Explanation:

- It shows a high-level view of the stakeholders (actors) of the system and how they interact with its functional requirements through use cases or scenarios
- Primary actors are on the left, while secondary ones are on the right
- There are some relationships, like “include”, that mean a function cannot happen without another one happening
- Colors are used to distinguish actors from each other to make it easy to know which actor is connected to which use case

3. Activity Diagram


Customer
System
Restaurant
Driver
Open app
Register / Log in
Validate credentials
Generate session token
Browse nearby restaurants
Apply filters (cuisine / rating / time)
Fetch restaurants by GPS location
Return filtered list
Select restaurant
View menu
Add items to cart
Review cart & total
Proceed to checkout
Enter payment details
Payment valid?
Yes
No
Display payment error
Retry payment
Process payment via gateway
Generate order ID
Send confirmation to customer
Notify restaurant of new order
Receive order notification
Confirm order
Prepare food
Mark order as Ready for Pickup
Query available drivers by location
Assign nearest driver
Send push notification to driver
Receive assignment notification



Model Explanation :

Why Do We Use Activity Diagram :

- It shows the system behavior step by step. Rather than describing the process in paragraphs, the diagram makes the sequence of actions immediately visible. Anyone reading it can follow the flow from top to bottom and understand exactly what happens at each stage.
- It shows who is responsible for each action. This is why swimlanes are used. Each vertical lane belongs to one actor: Customer, System, Restaurant, or Driver. When an action block appears in a lane, it means that actor is responsible for performing it. This makes responsibilities crystal clear with no ambiguity.
- It models decision points and alternative flows. The diamond shape in the diagram (Payment valid?) represents a decision. The diagram shows what happens in both the successful path (Yes) and the failure path (No). This directly corresponds to the alternative flows you wrote in your functional requirements.
- It connects your functional requirements to visual behavior. Every action block in this diagram maps directly to one of FRs. This is what makes the diagram academically valuable – it is not just a picture, it is a visual proof that your requirements are complete and logically ordered.

Customer Lane :

- Actions : Open app → Register/Log in → Browse restaurants → Apply filters → Select restaurant → View menu → Add items to cart → Review cart → Proceed to checkout → Enter payment details → (if payment fails) Display error and retry → Rate restaurant → Write review
- What this represents : Everything the human user does manually. These are the interactions that trigger the system to respond. This lane directly reflects FR-01 (Login), FR-02 (Browse), FR 03 (View Menu), FR-04 (Cart), FR-05 (Place Order), and FR-10 (Reviews).

System Lane :

- Actions : Validate credentials → Generate session token → Fetch restaurants by GPS → Return filtered list → Check if payment is valid → Process payment → Generate order ID → Send confirmation → Notify restaurant → Query available drivers → Assign nearest driver → Send push notification → Update order status to Delivered → Send delivery confirmation → Prompt for review → Save review → Update restaurant rating
- What this represents : Everything the software does automatically in the background without the user seeing it directly. This is the largest lane because the system orchestrates all other actors. This lane reflects FR-01, FR-02, FR-05, FR-06, FR-07, FR-09, FR-10, and FR-11.

Restaurant Lane :

- Actions : Receive order notification → Confirm order → Prepare food → Mark order as Ready for Pickup
- What this represents : The actions performed by the restaurant staff through their dashboard. This lane reflects FR-08 (Restaurant Dashboard & Menu Management). It is a short lane because the restaurant's role in the order flow is focused – they receive, confirm, prepare, and signal readiness.

Driver Lane :

- Actions : Receive assignment notification → Accept order → Travel to restaurant → Pick up order → Mark as Out for Delivery → Deliver to customer → Mark as Delivered
- What this represents : The physical and app-based actions performed by the delivery driver. This lane reflects FR-09 (Driver Assignment) and FR-06 (Real-Time Order Tracking) because each status update the driver makes feeds directly into the order tracking the customer sees.

Key Elements Explained :

- start and stop : These are the initial and final nodes of the diagram. start is represented by a filled black circle and marks the exact point where the process begins. stop is represented by a bullseye symbol and marks where the process ends. Every activity diagram must have exactly one start and at least one stop. •
- Action Blocks : action; Every line written between colons like :Place Order; represents a single action or activity. It is drawn as a rounded rectangle in the final diagram. Each one represents one concrete step performed by the actor in whose swim lane it appears.

- Decision Diamond if (Payment valid?) : This is the only branching point in your diagram. It represents a condition that can go two ways. The then (Yes) path continues the happy flow toward order confirmation. The else (No) path sends control back to the Customer lane where the error is shown and the payment is retried. After retrying, control returns to the System lane via endif and the flow continues. This models FR-07's alternative flow exactly.
- Swimlane Transitions : Every time the diagram switches from one lane to another, it means one actor has finished their part and handed off control to another. For example:
  - Customer enters payment details → hands off to System to validate
  - System assigns driver → hands off to Driver to act
  - Driver marks as Delivered → hands off to System to update records
  - System prompts for review → hands off to Customer to rate

These handoffs represent the integration points between your actors and are the most important parts of the diagram for showing how the system coordinates multiple parties simultaneously.